Phase-1

1A

```
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ rm -rf .pes
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ mkdir -p .pes/objects .pes/refs/heads
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
```

1B

```
All Phase 1 tests passed.
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
```

Phase -2

2A

```
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
```

2B

```
xxd "$TREE_OBJ" | head -20
Initialized empty PES repository in .pes/
Committed: 98e9ce3362bb... tmp
.pes/objects/f5/01eae1d87bbc75471ed69bbc1b32e89978f4dc3889f9d76a5ecd9edb5ef9aa
00000000: 7472 6565 2034 3600 3130 3036 3434 2074  tree 46.100644 t
00000010: 312e 7478 7400 2cf8 d83d 9ee2 9543 b34a  1.txt.,..=...C.J
00000020: 8772 7421 fdec b7e3 f3a1 83d3 3763 9025  .rt!........7c.%
00000030: de57 6db9 ebb4                           .Wm...
```

Phase -3

3A

```
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ rm -rf .pes file1.txt file2.txt
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt …
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)
```

3B

```
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776752896 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776752896 6 file2.txt
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
```

Phase -4

4A

```
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ ./pes log
commit 4ac0b754c73d24b1a93d90fa256eb3291216778346bf5a30332b458cb05afae1
Author: Sathvik  <PES2UG24CS430>
Date:   1776753121

    Add farewell

commit 28e4125aa0f2d16b2f0e0b416eaed647bda93ca548a23f071e53e9d6d9fd925c
Author: Sathvik  <PES2UG24CS430>
Date:   1776753121

    Add world

commit 1c098c77e1b5d396bdb1c5f05152975f5b0945a38ee5591d6af9201bbd1c2a07
Author: Sathvik  <PES2UG24CS430>
Date:   1776753121

    Initial commit

sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
```

4B

```
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/1c/098c77e1b5d396bdb1c5f05152975f5b0945a38ee5591d6af9201bbd1c2a07
.pes/objects/28/e4125aa0f2d16b2f0e0b416eaed647bda93ca548a23f071e53e9d6d9fd925c
.pes/objects/4a/c0b754c73d24b1a93d90fa256eb3291216778346bf5a30332b458cb05afae1
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
```

4C

```
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ cat .pes/refs/heads/main
cat .pes/HEAD
4ac0b754c73d24b1a93d90fa256eb3291216778346bf5a30332b458cb05afae1
ref: refs/heads/main
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
```

Final Submission

```
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$ make test-integration
Author: Sathvik  <PES2UG24CS430>
Date:   1776753354

    Add farewell

commit 9c090244c6f1329c60b40cda09d7d974e430ba3d2591b68c457c4e44788095cc
Author: Sathvik  <PES2UG24CS430>
Date:   1776753354

    Update file.txt

commit 3692f122f662d0a06534a082da02eb09a0f01252af25bf93f91f9dbf7ea58b05
Author: Sathvik  <PES2UG24CS430>
Date:   1776753354

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f68834ca9d55a5c4da99214975d153e5d24ba15bca8014b7a27204eb18642868

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/36/92f122f662d0a06534a082da02eb09a0f01252af25bf93f91f9dbf7ea58b05
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/9c/090244c6f1329c60b40cda09d7d974e430ba3d2591b68c457c4e44788095cc
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f6/8834ca9d55a5c4da99214975d153e5d24ba15bca8014b7a27204eb18642868

=== All integration tests completed ===
sathvik@sathvik-Standard-PC-Q35-ICH9-2009:~/OS_OR$
```

Phase 5: Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To switch branches, the system performs a handoff between internal records and your actual files. Inside the `.pes/` folder, the `HEAD` file is updated to point to the new branch path, and the "index" (the staging area) is reset to match that branch's specific version of files. In your working directory, the system deletes files that don't exist in the new branch and replaces others with versions pulled from the object store. This is complex because the system must act as a safety net; it has to ensure that any unsaved work you've done isn't accidentally deleted or overwritten during the transition.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

A conflict is detected by comparing a file's state in three places: your current work on disk, the last commit you made, and the branch you are trying to switch to. If the file on your disk is different from the version in your index or last commit, it is considered "dirty." If that same file is also different in the target branch compared to your current branch, the system sees a conflict. To prevent you from losing your unsaved changes, the checkout operation will stop and ask you to either commit or clear those changes first so that the incoming branch data doesn't wipe them out.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

A "Detached HEAD" occurs when the system points directly to a specific commit ID instead of a branch name. You can still make commits in this state, but they are "orphaned" because no branch name is keeping track of them. If you switch away to another branch, these commits seem to disappear because there is no pointer left to find them. To recover them, a user would look through the Reflog—a hidden history log that records every move the system makes—to find the lost commit's ID and then create a new branch at that location to save the work permanently.

Phase 6: Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

To clean up the object store, we use an algorithm called Mark-and-Sweep. In the "Mark" phase, the system starts at every branch and tag and follows their links to every related folder and file, adding their IDs to a Hash Set (a digital checklist that is very fast to search). In the "Sweep" phase, the system looks at every file in the object store; if an ID isn't on the checklist, it’s considered trash and deleted. For a repository with 100,000 commits, you would likely visit 500,000 to 1,000,000 objects, since each commit links to a root folder, sub-folders, and many individual files.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Running a cleanup while someone is still saving work creates a dangerous "race." Imagine you are currently saving a new file; the file is written to the store, but you haven't finished linking it to a commit object yet. If the cleanup starts at that exact second, it sees a file with no links, assumes it is garbage, and deletes it. When your commit tries to finalize, its data is gone, and the repository becomes corrupted. Git avoids this by using a grace period: it only deletes "trash" that is older than a certain time (like two weeks), giving new files plenty of time to get properly linked before they are even considered for deletion.

GitHub Link : https://github.com/Codemaster127-byte/os_orange_